

Sintaxe & Semântica

João Eduardo Montandon
DCC024

Introdução

G1.

```
<subexpr> ::= a | b | c | <subexpr> - <subexpr>
```

G2.

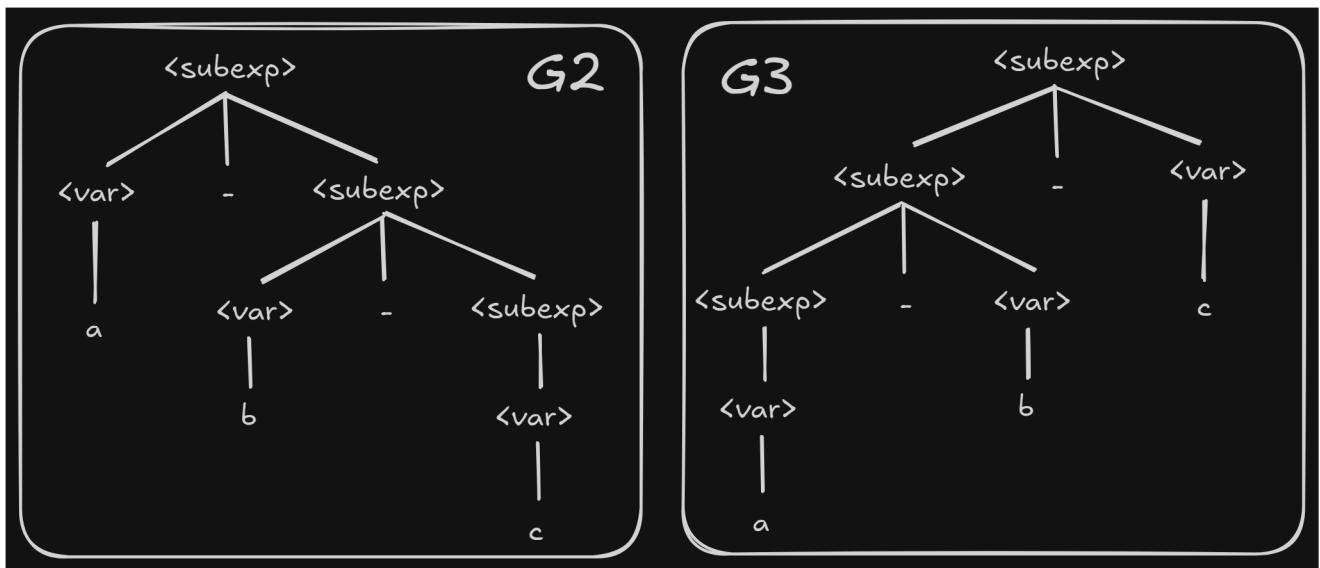
```
<subexpr> ::= <var> - <subexpr> | <var>  
<var> ::= a | b | c
```

G3.

```
<subexpr> ::= <subexp> - <var> | <var>  
<var> ::= a | b | c
```

Geram a mesma linguagem!

Mas elas **são iguais?** 🤔



Diferentes árvores, o que significa?

Diferentes árvores = diferentes computações

1 - 2 - 3 = -4 OU 1 - 2 - 3 = 2? 🤔

- Uma árvore de parsing impacta tanto a **sintaxe** quanto a **semântica** da linguagem que ela gera
- Isso torna o design de gramáticas algo muito mais difícil de se conseguir!

🎯 Obter árvores de parsing que sejam únicas && reflitam a semântica da linguagem

Estudo De Caso: Operadores

- Presente na maioria das linguagens de programação
- Varia conforme tipo da operação realizada (subtração, divisão, multiplicação e soma)
- **Operador**: refere-se tanto ao token (+ e *) quanto a operação em si
 - Geralmente pré-definidas, mas nem sempre

- **Operandos**: entrada da operação (números e variáveis)
- **Operadores unários** envolvem apenas um operando (~1)
- **Operadores binários** envolvem dois operandos (1 + 2)
- **Operadores ternários** envolvem três operandos (a ? b : c)

- *infix*: `a + b`
- *prefix*: `+ a b`
 - Outro exemplo: `++n`
- *postfix*: `a b +`
 - Outro exemplo: `n++`

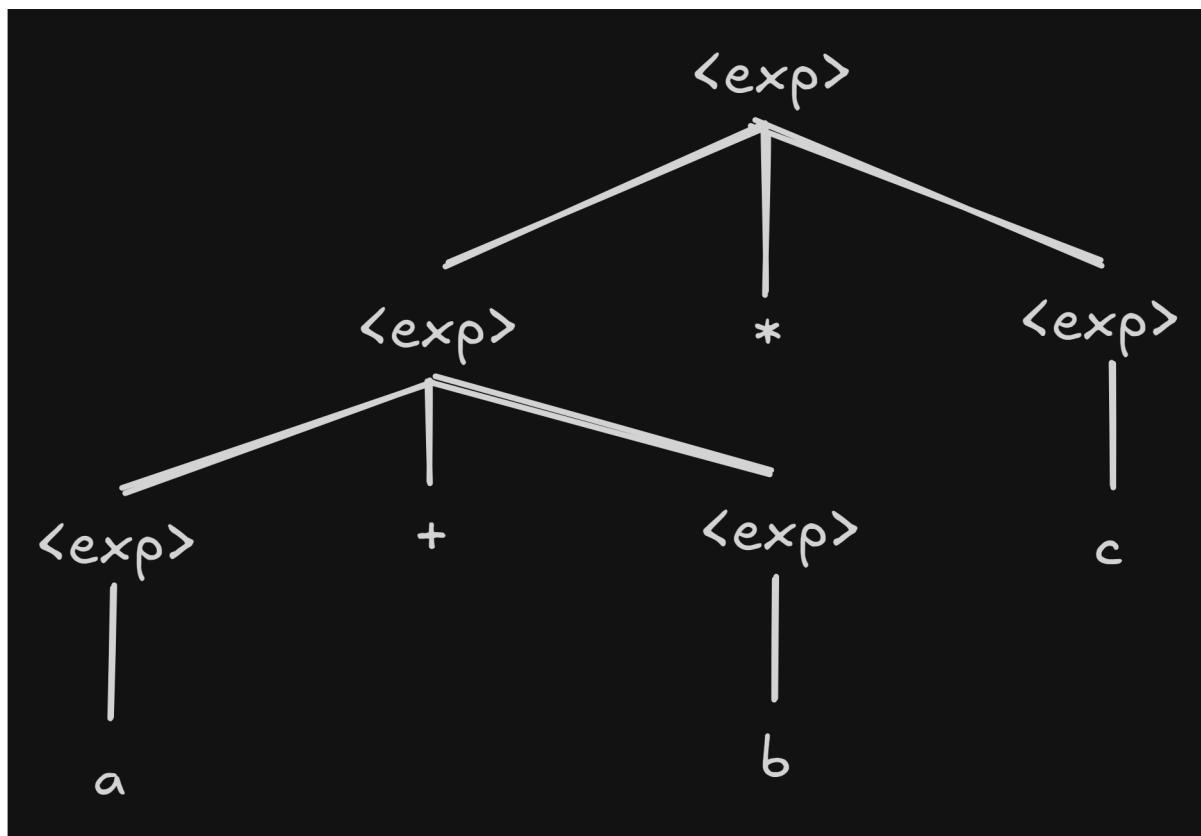
1. Precedência

G4.

```
<exp> ::= <exp> + <exp>
        | <exp> * <exp>
        | ( <exp> )
        | a | b | c
```

Quais inconsistências essa gramática possui?

`a + b * c = a + (b * c)` ou `(a + b) * c`? 🤔

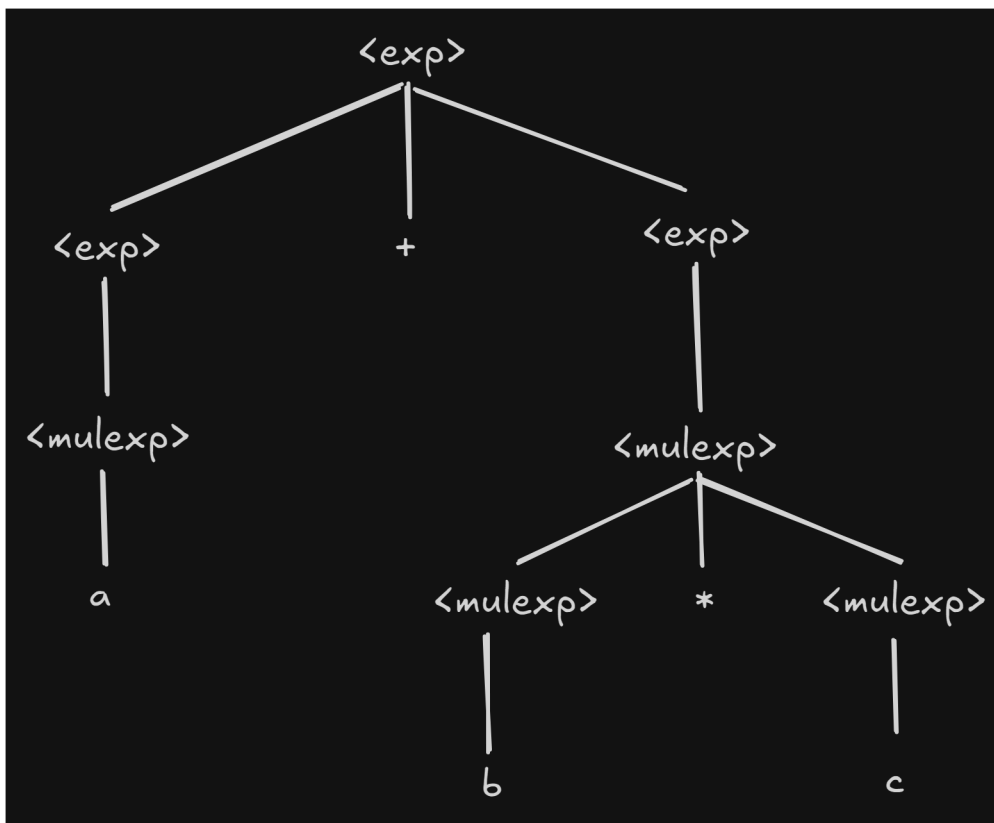


Como Corrigir?

G5.

```
<exp> ::= <exp> + <exp> | <mulexp>
<mulexp> ::= <mulexp> * <mulexp>
           | ( <exp> )
           | a | b | c
```

$a + b * c = \dots ?$



- Intuitivamente: Operadores de maior precedência são computados antes dos de menor precedência.
 - Tecnicamente: Operadores de maior precedência não podem gerar operadores de menor precedência
-

Algumas Curiosidades

- C: 15 níveis de precedência
- Pascal: 5 níveis de precedência
- Smalltalk: um nível de precedência

Menos é mais? 🤔

2. Associatividade

G5.

```
<exp> ::= <exp> + <exp> | <mulexp>
<mulexp> ::= <mulexp> * <mulexp>
           | ( <exp> )
           | a | b | c
```

`a + b + c = ...`

→ `(a + b) + c`

→ `a + (b + c)`

- 🎯 Uma gramática deve sempre gerar apenas uma árvore de parsing!
- Neste caso, precedência se dá pela **ordem** (*left associative*)

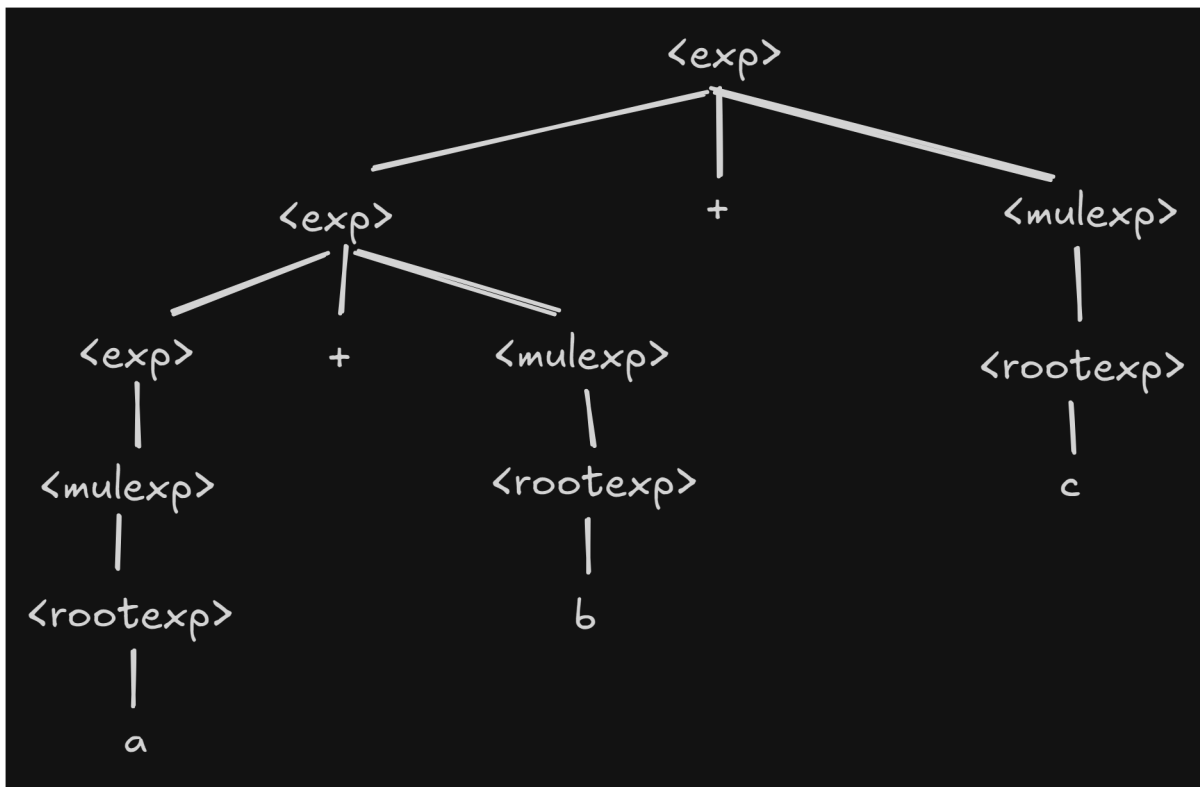
`a + b + c = (a + b) + c`

Como implementar?

G6.

```
<exp> ::= <exp> + <mulexp> | <mulexp>
<mulexp> ::= <mulexp> * <rootexp> | <rootexp>
<rootexp> ::= ( <exp> )
           | a | b | c
```

`a + b + c = ... ?`



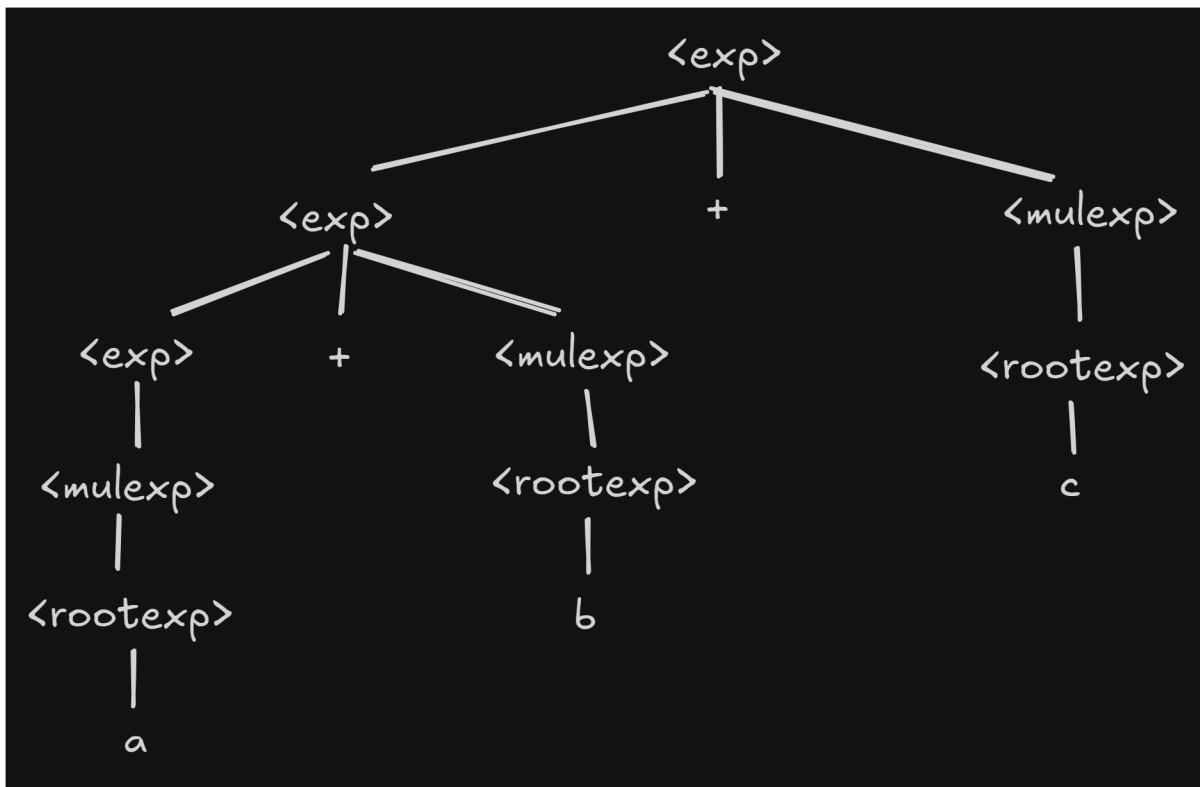
- Intuitivamente: Os operadores são computados da esquerda para a direita
 - Árvore "pende" para a esquerda
- Tecnicamente: Operadores de mesmo nível são gerados sempre a esquerda
 - *left associative*

Algumas Curiosidades

- Alguns operadores são **right associative**
 - Java: `a = b = 1`
 - ML: `1::2::[3,4]` → `[1, 2, 3, 4]`
- Em algumas linguagens, existem operadores não-associativos
 - `1 < 2 < 3` ❌
 - `1 < (2 < 3)` ✅

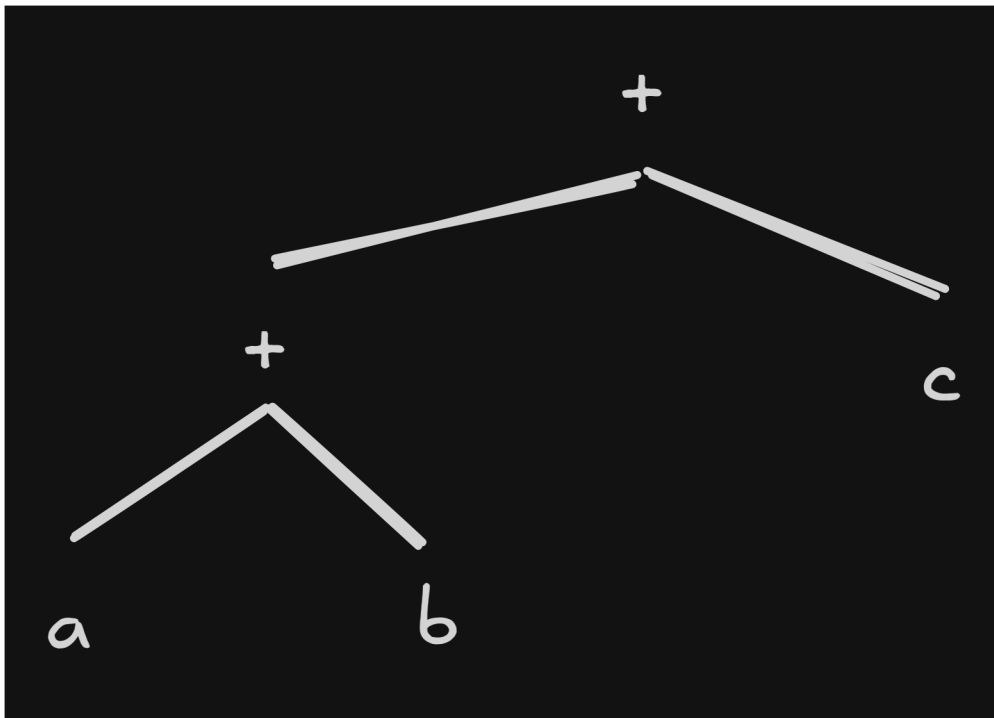
AST

Uma vez processada, muitos dos nós da árvore de parsing perdem sua finalidade



-
- Que importância `<rootexp>` tem para o programa?
 - O programa precisa saber que `a` foi gerado a partir de três nós não-terminais?
-

- Árvore de Sintaxe Abstrata (AST): Versão compacta de uma árvore de parsing
 - Varia conforme linguagem
 - Utilizada como uma representação interna do programa
 - Característica: Todo nó possui uma função dentro da linguagem
-



Conclusões

- Gramáticas definem mais do que a sintaxe da linguagem
- Ao determinar como a árvore de parsing será gerada, **determinamos a ordem de computação da instrução!**